

Data Mastery 120: Roadmap Integral de Minería y Análisis de Datos

Día 1 – Conexión y Carga (Dataset: IBM HR)



Objetivo del Día 1

- Conectar el **dataset IBM HR** a tu **entorno SQL** (PostgreSQL + DBeaver).
- Explorar las primeras **filas** para **validar** la **carga**.
- Construir un **esquema ER inicial** del dataset.



Normalización de Columnas (snake_case)

Antes de cargar el **dataset** en **SQL**, **normalizamos** los **nombres** de las **columnas** para **evitar conflictos** de **mayúsculas/minúsculas**.

"""

TÍTULO: Pipeline de Normalización y Limpieza de Datos de RRHH

OBJETIVO: Automatizar la lectura de reportes de Recursos Humanos, estandarizar los

nombres de las columnas a formato 'snake_case' y exportar una versión limpia con

marca de tiempo, asegurando la compatibilidad con Excel y el orden histórico.

"""

Herramientas para el manejo de datos y archivos CSV

import pandas as pd

import csv

Utilidades para interactuar con las carpetas y rutas del sistema operativo

import os

from pathlib import Path

Funciones para limpieza de texto (expresiones regulares) y gestión de fechas

import re

from datetime import datetime

1. Configuración de rutas.

--- Configuración de rutas del proyecto ---

Localiza la carpeta raíz subiendo tres niveles y define las rutas de entrada (raw)

y salida (processed) para mantener los datos organizados.

base_dir = Path(__file__).resolve().parents[2]

target_raw_dir = base_dir / '02_data' / 'raw' / 'rrhh'

cleaned_target_dir = base_dir / '02_data' / 'processed' / 'rrhh'

--- Identificación del archivo de origen ---

Define el nombre específico del archivo de RRHH y construye su ruta

completa para que el sistema pueda localizarlo.

```

rrhh_file_name = 'WA_Fn-UseC_-HR-Employee-Attrition'
rrhh_path_csv = os.path.join(target_raw_dir, f'{rrhh_file_name}.csv')
# --- Generación del archivo de salida con marca de tiempo ---
# Crea un nombre único para el archivo procesado usando la fecha y hora
actual.
# Esto evita que los resultados nuevos sobrescriban a los anteriores.
time_stamp = datetime.now().strftime('%Y%m%d_%H%M%S')
cleaned_file_name = f'rrhh_limpio_{time_stamp}.csv'
path_csv = os.path.join(cleaned_target_dir, cleaned_file_name)

# -----
# 2. Normalización de Nombres de Columnas.
# -----
def to_snake_case(name):
    """
    Convierte un texto de formato CamelCase o PascalCase a snake_case.
    """
    # Inserta un guion bajo entre una letra minúscula y una mayúscula que
    # inicia una palabra.
    # Ejemplo: "EmpleadoId" -> "Empleado_Id"
    s1 = re.sub('([A-Z][a-z]+)', r'\1_\2', name)
    # Maneja casos con números o mayúsculas consecutivas y convierte todo
    # a minúsculas.
    # Ejemplo: "Empleado_Id" -> "empleado_id"
    return re.sub('([a-z0-9])([A-Z])', r'\1_\2', s1).lower()

# -----
# 3. Carga del Dataset.
# -----
try:
    # --- Carga y normalización de datos ---
    # Importamos el archivo CSV a un formato de tabla (DataFrame) y
    # transformamos
    # todos los nombres de las columnas a 'snake_case' para estandarizar el
    # formato.
    df = pd.read_csv(rrhh_path_csv)
    df.columns = [to_snake_case(col) for col in df.columns]
    # --- Preparación para la exportación ---
    # Convertimos la tabla en una lista de diccionarios y extraemos los
    # nombres
    # de las columnas; esto facilita la escritura posterior del archivo limpio.
    datos_dict = df.to_dict(orient='records')
    mis_cabeceras = df.columns.tolist()
except Exception as e:

```

Gestión de errores en caso de que el archivo no exista o el formato sea incorrecto.

```
print(f"❌ Error al cargar o procesar el dataset: {e}")
```

```
datos_dict = None
```

4. Exportación de Datos.

```
def exportar_a_csv(ruta_destino, datos, cabeceras):
```

```
    """
```

Gestiona la creación de carpetas y guarda los datos procesados en un archivo CSV.

```
    """
```

```
    if datos is None:
```

```
        return
```

```
    try:
```

```
        # --- Preparación del entorno ---
```

```
        # Verifica si existe la carpeta de destino; si no, la crea automáticamente
```

```
        # para evitar errores al intentar guardar el archivo.
```

```
        if not os.path.exists(cleaned_target_dir):
```

```
            os.makedirs(cleaned_target_dir, exist_ok=True)
```

```
            print(f"✅ Carpeta creada en: {cleaned_target_dir}")
```

```
        # --- Escritura del archivo ---
```

```
        # Abre el archivo con codificación 'utf-8-sig' para que Excel reconozca
```

```
        # correctamente tildes y eñes. Se usa el delimitador '|' para mayor
```

claridad.

```
        with open(ruta_destino, mode='w', newline="", encoding='utf-8-sig') as  
archivo_csv:
```

```
            df.to_csv(archivo_csv, sep='|', index=False)
```

```
        # --- Confirmación y resumen ---
```

```
        # Muestra en consola un resumen técnico de los datos guardados y
```

```
        # confirma la ubicación final del archivo.
```

```
        print(df.info())
```

```
        print(f"\n✅ Proceso completado exitosamente.")
```

```
        print(f"📁 Archivo guardado en: {ruta_destino}")
```

```
        print(f"📄 CSV: {cleaned_file_name}")
```

```
    except PermissionError:
```

Error común si el archivo de destino está abierto en Excel o por otro usuario.

```
    print(f"❌ Error de Permiso: El archivo '{ruta_destino}' está abierto en  
otro programa.")
```

```
    except Exception as e:
```

```

    # Captura cualquier otro imprevisto durante la exportación.
    print(f" ✖ Ocurrió un error inesperado al exportar: {e}")
# --- Punto de entrada del script ---
# Asegura que el proceso de limpieza y exportación solo se inicie si el
# archivo se ejecuta directamente (no si se importa desde otro script).
if __name__ == "__main__":
    print("--- Iniciando proceso de normalización ---\n")
    exportar_a_csv(path_csv, datos_dict, mis_cabeceras)

```

Ejemplo de columnas normalizadas:

- EmployeeNumber → employee_number
- Attrition → attrition
- JobRole → job_role
- MonthlyIncome → monthly_income
- PerformanceRating → performance_rating
- YearsAtCompany → years_at_company
- EducationField → education_field

Guía Detallada Paso a Paso

Paso 1. Instalación de PostgreSQL & DBeaver

- Descarga **PostgreSQL** desde el sitio oficial de **PostgreSQL** (<https://www.postgresql.org/download/>).
- Instala **DBeaver** para administrar la base (<https://dbeaver.io/download/>).

Paso 2. Preparación del Entorno

Abrir **DBeaver** → Cliente gráfico para gestionar **PostgreSQL**.

Crear conexión a **PostgreSQL**:

- Menú: **Database** → **New Database Connection**.
- Selecciona **PostgreSQL**.
- Configura:
 - **Host**: localhost
 - **Port**: 5432
 - **Database**: postgres (por defecto)
 - **User**: postgres
 - **Password**: tu contraseña definida.
- Haz clic en **Test Connection** → debe mostrar **Success**.
- **Guarda** la conexión.



Paso 3. Crear la Base de Datos ibm_hr

En el editor **SQL** de **DBeaver**:

- Abre una nueva consulta y ejecuta:

```

/*
-----
TÍTULO: Creación de la base de datos para Recursos Humanos de IBM
OBJETIVO: Crear un contenedor lógico (base de datos) para almacenar
toda la
        información relacionada con el personal, departamentos y
nómina.
DESCRIPCIÓN: Este script inicializa el entorno de trabajo ejecutando la
sentencia
        DDL (Data Definition Language) necesaria para generar el
esquema
        principal de la base de datos 'ibm_hr'.
-----
*/

-- 1. Sentencia principal para crear la base de datos
-- Esta línea genera una nueva base de datos vacía llamada 'ibm_hr'.
CREATEDATABASE ibm_hr;

-- NOTA OPERATIVA: Una vez creada, asegúrate de activar la base de
datos
-- (Set as default) para que todas las tablas posteriores se guarden en
este proyecto.

```

- **Check de Ingeniería:**
 - Confirmar que el nombre está en **snake_case**.
 - Validar que la base aparece en el panel izquierdo.



Paso 4. Crear la Tabla Principal employee_master_data

Selecciona la base **ibm_hr**.

Ejecuta el script de creación de tabla con columnas normalizadas:

```

/*****
*****
TÍTULO: Creación de Tabla Maestra de Empleados
OBJETIVO: Establecer la estructura base para almacenar la información
demográfica,

```

organizacional y de compensación de los colaboradores.
DESCRIPCIÓN: Este script crea la tabla 'employee_master_data' si no existe,

incluyendo campos clave para análisis de retención, movilidad, estructura de puestos y compensaciones (sueldos y opciones).

*****/

CREATE TABLE IF NOT EXISTS employee_master_data(

--

=====

=====

-- 1. DATOS DEMOGRÁFICOS Y PERSONALES

-- Información básica del colaborador.

--

=====

=====

age INT NOT NULL, -- Edad del colaborador (obligatorio).

gender VARCHAR(50), -- Género registrado.

marital_status VARCHAR(20), -- Estado civil (Single, Married, Divorced).

over18 CHAR(1), -- Verificación de mayoría de edad (Y/N).

--

=====

=====

-- 2. INDICADORES DE RETENCIÓN Y MOVILIDAD

-- Métricas clave para análisis de fuga de talento (Attrition) y traslados.

--

=====

=====

attrition VARCHAR(10), -- Abandono de la empresa (Yes/No).

business_travel VARCHAR(50), -- Frecuencia de viajes laborales.

distance_from_home INT, -- Distancia al centro de trabajo en km.

--

=====

=====

-- 3. ESTRUCTURA ORGANIZACIONAL Y PUESTO

-- Ubicación funcional del empleado.

--

=====

=====

department VARCHAR(50), -- Área o departamento funcional.

```

job_role VARCHAR(50),          -- Nombre del cargo o función.
job_level INT,                 -- Nivel de jerarquía en la organización.
employee_number INT PRIMARY KEY, -- Identificador único (Llave
Primaria).
employee_count INT,           -- Conteo unitario (usualmente 1 por
fila).

--
=====
=====
-- 4. COMPENSACIÓN Y BENEFICIOS
-- Datos financieros relacionados con el empleo.
--
=====
=====
monthly_income INT,            -- Sueldo mensual bruto.
monthly_rate INT,              -- Tasa o costo mensual relativo.
daily_rate INT,                -- Pago por jornada diaria.
hourly_rate INT,               -- Pago por hora laborada.
percent_salary_hike INT,       -- Porcentaje del último incremento.
stock_option_level INT,        -- Nivel de opciones sobre acciones.

--
=====
=====
-- 5. Educación y Formación
-- Historial académico y preparación técnica.
--
=====
=====
education INT,                 -- Nivel académico alcanzado (codificado).
education_field VARCHAR(50),   -- Especialidad o carrera
estudiada.
training_times_last_year INT,  -- Cursos de capacitación tomados
el año pasado.

--
=====
=====
-- 6. Encuestas de Satisfacción y Desempeño
-- Percepción del empleado y calidad de su trabajo (Escala
numéricas).
--
=====
=====

```



```

environment_satisfaction INT,      -- Satisfacción con el entorno
laboral.
job_involvement INT,              -- Nivel de compromiso con el puesto.
job_satisfaction INT,             -- Nivel de felicidad en el trabajo.
performance_rating INT,           -- Calificación de la última evaluación.
relationship_satisfaction INT,     -- Calidad de relación con colegas.
work_life_balance INT,            -- Equilibrio entre vida y trabajo.

--
=====
=====
-- 7. Trayectoria y Permanencia
-- Experiencia acumulada dentro y fuera de la organización.
--
=====
=====
num_companies_worked INT,          -- Cantidad de empresas previas.
total_working_years INT,           -- Años totales de experiencia laboral.
years_at_company INT,              -- Antigüedad en la empresa actual.
years_in_current_role INT,         -- Tiempo en el puesto actual.
years_since_last_promotion INT,    -- Años desde el último ascenso.
years_with_curr_manager INT,       -- Tiempo bajo el mando del jefe
actual.

--
=====
=====
-- 8. Condiciones Laborales
-- Parámetros finales sobre la jornada de trabajo.
--
=====
=====
over_time VARCHAR(10),             -- Realización de horas extra
(Yes/No).
standard_hours INT-- Horas base establecidas por contrato.
);

```

- **Pitfall común:** olvidar ";" al final o usar **nombres con espacios**.
- **Check de Ingeniería:** todos los nombres en **snake_case**, tipos de datos correctos.



Paso 5. Importar el Dataset IBM HR (Bulk Load)

Antes de **normalizar**, necesitamos los **datos** dentro. Usaremos el **asistente de DBeaver** para mayor precisión:

Opción 1:

1. Menú: **Tools** → **Data Transfer** → **Import Data**.
2. Selecciona **archivo CSV** del dataset **IBM HR**.
3. Destino: **tabla employee_master_data**.
4. Configura **delimitador** , y **encabezados**.
5. **Ejecuta** la **importación**.

Opción 2:

1. En **DBeaver**, haz clic derecho sobre la **tabla employee_master_data** creada ayer.
2. Selecciona **"Import Data"** -> **CSV**.
3. Busca tu **archivo de IBM HR**.
4. **Mapeo**: Asegúrate de que las **columnas** del CSV coincidan con los nombres de la tabla.
Ejecutar: Una vez finalizado, **verifica** que los **datos** se **cargaron** correctamente.



Paso 6. Controles de la calidad de los datos (employee_master_data)

- **Confirma que los datos se cargaron correctamente:**

```
/*****  
*****/
```

Título: Visualización Preliminar de Datos Maestros de Empleados
Objetivo: Obtener una muestra rápida de los datos almacenados en la tabla

principal de empleados.

Descripción: Selecciona las primeras 10 filas de la tabla 'employee_master_data'

para verificar la estructura, el tipo de datos y la calidad de la información sin cargar toda la base de datos.

```
*****
*****/
```

```
-- Selecciona todas las columnas (*) de la tabla especificada
SELECT *
-- Define la tabla de origen de los datos
FROM employee_master_data
-- Limita el resultado a solo las primeras 10 filas para una vista rápida
LIMIT 10;
```

	age	gender	marital_status	over18	attrition	business_travel	distance_from_home	department	job
1	41	Female	Single	Y	Yes	Travel_Rarely		Sales	Sales Executive
2	49	Male	Married	Y	No	Travel_Frequently		Research & Development	Research Scientist
3	37	Male	Single	Y	Yes	Travel_Rarely		Research & Development	Laboratory Assistant
4	33	Female	Married	Y	No	Travel_Frequently		Research & Development	Research Scientist
5	27	Male	Married	Y	No	Travel_Rarely		Research & Development	Laboratory Assistant
6	32	Male	Single	Y	No	Travel_Frequently		Research & Development	Laboratory Assistant
7	59	Female	Married	Y	No	Travel_Rarely		Research & Development	Laboratory Assistant

○ Detección de registros duplicados:

```
/*
*****/

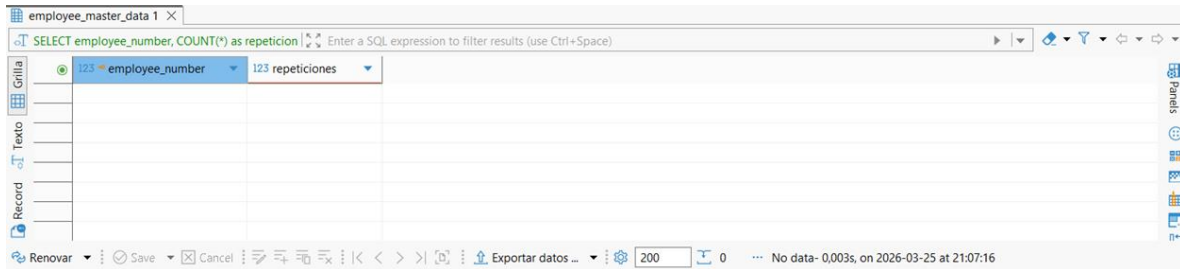
TÍTULO: Detección de Duplicados en Maestro de Empleados
OBJETIVO: Identificar números de empleado que aparecen más de una
vez en la tabla
maestra para asegurar la integridad de los datos.
DESCRIPCIÓN: Este script consulta la tabla principal de empleados,
agrupa por el
identificador único y filtra aquellos grupos que tienen un conteo
superior a 1, indicando una duplicidad de registros.

*****/
```

```
SELECT
-- Selecciona el identificador del empleado
employee_number,
-- Cuenta cuántas veces aparece cada número para identificar el
total de duplicados
COUNT(*) AS repeticiones
-- Define la tabla de origen de datos maestros de empleados
FROM employee_master_data
-- Agrupa los resultados por número de empleado para consolidar el
conteo
GROUP BY employee_number
```

-- Filtro final: Solo muestra los números de empleado cuyo conteo sea mayor a 1

HAVING COUNT(*) > 1;



The screenshot shows a data grid interface with a table titled 'employee_master_data 1'. The table has two columns: 'employee_number' and 'repeticiones'. The 'employee_number' column has a dropdown menu showing '123' and 'repeticiones'. The 'repeticiones' column has a dropdown menu showing '123'. The table is empty. The interface includes a search bar, a filter icon, and a status bar at the bottom.

○ Identificación de Valores Nulos en Columnas Críticas:

```
/*
*****
TITULO: Depuración de Maestros de Empleados
OBJETIVO: Obtener una lista limpia de colaboradores desde la tabla
maestra.
DESCRIPCIÓN: Selecciona todos los registros de
'employee_master_data' que tengan
información crítica mínima, excluyendo registros donde el ID,
departamento, puesto o salario estén vacíos (NULL).
*****
*/
```

SELECT *

-- Tabla fuente con la información cruda de los colaboradores

FROM employee_master_data

-- CRITERIOS DE VALIDACIÓN: El registro debe tener contenido en AL MENOS

-- uno de estos campos para ser considerado válido y útil.

WHERE

employee_number IS NOT NULL OR-- 1. Que el ID de empleado esté definido.

department IS NOT NULL OR-- 2. Que el departamento tenga un nombre asignado.

job_role IS NOT NULL OR-- 3. Que el puesto de trabajo esté definido.

monthly_income IS NOT NULL;-- 4. Que exista un registro del salario mensual.

employee_master_data 1

SELECT * FROM employee_master_data WHERE em

Enter a SQL expression to filter results (use Ctrl+Space)

	123 age	AZ gender	AZ marital_status	AZ over18	AZ attrition	AZ business_travel	123 distance_from_home	AZ department	AZ job_title
1	41	Female	Single	Y	Yes	Travel_Rarely		1 Sales	Sales Executive
2	49	Male	Married	Y	No	Travel_Frequently		8 Research & Development	Research Scientist
3	37	Male	Single	Y	Yes	Travel_Rarely		2 Research & Development	Laboratory Technician
4	33	Female	Married	Y	No	Travel_Frequently		3 Research & Development	Research Scientist
5	27	Male	Married	Y	No	Travel_Rarely		2 Research & Development	Laboratory Technician
6	32	Male	Single	Y	No	Travel_Frequently		2 Research & Development	Laboratory Technician
7	59	Female	Married	Y	No	Travel_Rarely		3 Research & Development	Laboratory Technician

1470 row(s) fetched - 0.03s (0.007s fetch), on 2026-03-25 at 21:10:30

○ Auditoría de Integridad (Valores fuera de rango o vacíos):

```

/*****
*****
TITULO: Detección de Colaboradores con Datos Incompletos o
Inválidos
OBJETIVO: Identificar empleados en la tabla maestra que carecen de
área asignada
o que tienen un sueldo ilógico (cero o negativo).
DESCRIPCIÓN: Este script consulta la base de datos de RRHH para
encontrar
registros con errores críticos en 'departamento' o 'ingreso
mensual'
que requieren corrección.
*****/

```

SELECT

```

employee_number, -- Identificador único del colaborador
department,      -- Área asignada (se busca si está vacía)
monthly_income-- Sueldo registrado (se busca si es incorrecto)
-- Tabla principal que contiene la información de RRHH

```

FROM employee_master_data

```

-- CRITERIOS DE ERROR: Filtra registros con problemas de calidad de
datos

```

WHERE

```

department= ''OR-- Error 1: Departamento vacío (texto sin
contenido)
monthly_income <= 0;-- Error 2: Salario cero o negativo (valor
ilógico)

```



```

/*****
*****
TITULO: Reporte de Conteo de Empleados por Departamento
OBJETIVO: Obtener el total de empleados activos agrupados por su
área funcional.
DESCRIPCIÓN: Este script consulta la tabla maestra de empleados para
sumar cuántos
empleados pertenecen a cada departamento, permitiendo
visualizar la
distribución del personal.
*****/

```

SELECT

-- Selecciona la columna del departamento para identificar el grupo

department,

-- Cuenta el número total de filas (empleados) dentro de cada grupo

COUNT(*) AS total_empleados

-- Define la tabla principal donde se almacena la información de los empleados

FROM employee_master_data

GROUP BY

-- Agrupa los resultados por departamento para que el conteo no sea global,

-- sino específico para cada área definida en la tabla.

department;

department	total_empleados
Human Resources	63
Research & Development	961
Sales	446

- **Pitfall común:** olvidar **GROUP BY** cuando usas agregaciones.
- **Check de Ingeniería:** validar que los resultados coinciden con expectativas (ej. número total de empleados).

Paso 7: Identificación de Redundancia (Análisis)

Observa la columna **Department** y **JobRole**. El nombre "**Sales**" se repite cientos de veces. Si mañana la empresa decide llamar a "**Sales**" como "**Commercial**", tendrías que actualizar miles de filas. **Solución: Crear tablas maestras.**

Paso 8: Creación de Tablas Maestras (Normalización)

Copia este bloque y ejecútalo primero. Estas tablas guardarán los nombres únicos de departamentos, roles y áreas de estudio.

Las **Tablas Maestras** son fundamentales en bases de datos relacionales para almacenar datos estáticos (ej. clientes, productos, usuarios). Eliminan redundancia y aseguran integridad referencial, evitando repetir información en tablas transaccionales.

```
-----  
-----  
-- TÍTULO: Creación de Catálogo de Departamentos  
-- OBJETIVO: Definir la estructura base para almacenar los  
departamentos de la empresa.  
-- DESCRIPCIÓN: Este script crea una tabla maestra  
(`department_catalog`) que  
--          servirá para categorizar empleados o áreas, garantizando que  
--          cada departamento tenga un nombre único y un identificador  
propio.  
-----  
-----  
  
-- Bloque de creación de la tabla  
CREATE TABLE department_catalog (  
    -- Define el ID del departamento como clave primaria,  
    autoincrementable.  
    department_id SERIAL PRIMARY KEY,  
    -- Define el nombre del departamento: texto hasta 50 caracteres,  
    -- obligatorio (NOT NULL) y sin repetirse (UNIQUE).  
    department_name VARCHAR (50) UNIQUE NOT NULL  
);
```


Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,048s
Start time	Sun Mar 29 19:14:18 CST 2026
Finish time	Sun Mar 29 19:14:18 CST 2026
Query	-- Bloque de creación de la tabla
	CREATE TABLE department_catalog (
	-- Define el ID del departamento como clave primaria, autoincrementable.
	department_id SERIAL PRIMARY KEY,
	-- Define el nombre del departamento: texto hasta 50 caracteres,
	-- obligatorio (NOT NULL) y sin repetirse (UNIQUE).
	department_name VARCHAR (50) UNIQUE NOT NULL
	)

-- TÍTULO: Creación de Catálogo de Roles de Trabajo

-- OBJETIVO: Definir una estructura base para almacenar los diferentes puestos

-- o roles laborales disponibles en la organización.

-- DESCRIPCIÓN: Este script crea la tabla 'job_role_catalog', asegurando que

-- cada rol tenga un identificador único y un nombre no duplicado.

-- Creación de la tabla que funcionará como catálogo maestro de roles

CREATE TABLE job_role_catalog(

-- Identificador único autoincremental para cada rol (llave primaria)

job_role_id SERIAL PRIMARY KEY,

-- Nombre del rol (ej. "Gerente", "Analista"), máximo 50 caracteres,

-- no permite duplicados y es obligatorio

job_role_name VARCHAR (50) UNIQUE NOT NULL

);

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,013s
Start time	Sun Mar 29 19:27:45 CST 2026
Finish time	Sun Mar 29 19:27:45 CST 2026
Query	-- Creación de la tabla que funcionará como catálogo maestro de roles
	CREATE TABLE job_role_catalog(
	-- Identificador único autoincremental para cada rol (llave primaria)
	job_role_id SERIAL PRIMARY KEY,
	-- Nombre del rol (ej. "Gerente", "Analista"), máximo 50 caracteres,
	-- no permite duplicados y es obligatorio
	job_role_name VARCHAR (50) UNIQUE NOT NULL
	)

/*****

TÍTULO: Creación del Catálogo de Campos Educativos

OBJETIVO:

Crear una tabla de referencia (catálogo) para almacenar las diferentes áreas, disciplinas o campos educativos.

DESCRIPCIÓN:

Esta tabla servirá como una tabla maestra para asegurar la integridad referencial en otras tablas (por ejemplo, en la tabla de empleados o estudiantes), garantizando que solo se utilicen campos educativos predefinidos.

*****/
*****/

-- Se crea la tabla 'education_field_catalog' para almacenar el catálogo de campos.

```
CREATE TABLE education_field_catalog(  
  -- Define la clave primaria. 'SERIAL' crea un número entero  
  -- autoincremental único.  
  education_field_id SERIAL PRIMARY KEY,  
  -- Define el nombre del campo educativo.  
  -- VARCHAR(50): Permite texto de hasta 50 caracteres.  
  -- UNIQUE: Asegura que no haya dos campos educativos con el  
  -- mismo nombre.  
  -- NOT NULL: Obliga a que el campo siempre tenga un valor (no  
  -- permite nulos).  
  education_field_name VARCHAR (50) UNIQUE NOT NULL  
);
```

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,016s
Start time	Sun Mar 29 19:32:39 CST 2026
Finish time	Sun Mar 29 19:32:39 CST 2026
Query	-- Se crea la tabla 'education_field_catalog' para almacenar el catálogo de campos. CREATE TABLE education_field_catalog(-- Define la clave primaria. 'SERIAL' crea un número entero autoincremental único. education_field_id SERIAL PRIMARY KEY, -- Define el nombre del campo educativo. -- VARCHAR(50): Permite texto de hasta 50 caracteres. -- UNIQUE: Asegura que no haya dos campos educativos con el mismo nombre. -- NOT NULL: Obliga a que el campo siempre tenga un valor (no permite nulos).



Paso 9: Poblado de Catálogos Maestros desde Datos de Empleados:

```

/*****
****
Título: Poblado de Catálogos Maestros desde Datos de Empleados
Objetivo: Normalizar la base de datos extrayendo valores únicos de
departamentos,
roles y campos educativos para llenar sus respectivas tablas de
catálogo.
Descripción: Este script lee la tabla 'employee_master_data' y toma los
valores
distintos de las columnas department, job_role y education_field,
insertándolos en nuevas tablas de catálogo para evitar
duplicidad y
mejorar la integridad de los datos.
*****/

-- Bloque 1: Poblar el catálogo de departamentos
-- Extrae nombres de departamentos únicos de la tabla maestra y los
inserta en 'department_catalog'.
INSERT INTO department_catalog (department_name)
SELECT DISTINCT department-- Selecciona departamentos distintos
FROM employee_master_data-- Desde la tabla origen
WHERE department IS NOT NULL-- Filtra nulos
AND department <> ''-- Filtra cadenas vacías

```

Estadísticas 1	
Name	Value
Updated Rows	3
Execute time	0,018s
Start time	Sun Mar 29 19:39:30 CST 2026
Finish time	Sun Mar 29 19:39:30 CST 2026
Query	-- Bloque 1: Poblar el catálogo de departamentos
	-- Extrae nombres de departamentos únicos de la tabla maestra y los inserta en 'department_catalog'.
	INSERT INTO department_catalog (department_name)
	SELECT DISTINCT department-- Selecciona departamentos distintos
	FROM employee_master_data-- Desde la tabla origen
	WHERE department IS NOT NULL -- Filtra nulos
	AND department <> ''

```

-- Bloque 2: Poblar el catálogo de roles laborales (puestos)
-- Extrae los roles únicos de la tabla maestra para alimentar la tabla
'job_role_catalog'.
INSERT INTO job_role_catalog (job_role_name)
SELECT DISTINCT job_role-- Selecciona puestos únicos

```

FROM employee_master_data-- Desde la tabla origen
WHERE job_role **IS NOT NULL**-- Filtra nulos
AND job_role <> ''-- Filtra cadenas vacías

Estadísticas 1	
Name	Value
Updated Rows	9
Execute time	0,006s
Start time	Sun Mar 29 19:43:54 CST 2026
Finish time	Sun Mar 29 19:43:54 CST 2026
Query	-- Bloque 2: Poblar el catálogo de roles laborales (puestos) -- Extrae los roles únicos de la tabla maestra para alimentar la tabla 'job_role_catalog'. INSERT INTO job_role_catalog (job_role_name) SELECT DISTINCT job_role-- Selecciona puestos únicos FROM employee_master_data-- Desde la tabla origen WHERE job_role IS NOT NULL-- Filtra nulos AND job_role <> ''
CST es Editable Inserción inteligente 87 : 8 : 4331 Set: 0 0	

-- Bloque 3: Poblar el catálogo de áreas educativas
-- Extrae los campos de estudio únicos de la tabla maestra para la tabla 'education_field_catalog'.
INSERT INTO education_field_catalog(education_field_name)
SELECT DISTINCT education_field-- Selecciona áreas educativas únicas
FROM employee_master_data-- Desde la tabla origen
WHERE education_field **IS NOT NULL**-- Filtra nulos
AND education_field <> ''-- Filtra cadenas vacías

Estadísticas 1	
Name	Value
Updated Rows	6
Execute time	0,006s
Start time	Sun Mar 29 19:49:15 CST 2026
Finish time	Sun Mar 29 19:49:15 CST 2026
Query	-- Bloque 3: Poblar el catálogo de áreas educativas -- Extrae los campos de estudio únicos de la tabla maestra para la tabla 'education_field_catalog'. INSERT INTO education_field_catalog(education_field_name) SELECT DISTINCT education_field-- Selecciona áreas educativas únicas FROM employee_master_data-- Desde la tabla origen WHERE education_field IS NOT NULL-- Filtra nulos AND education_field <> ''
CST es Editable Inserción inteligente 93 : 10 : 4628 Set: 0 0	



Paso 10: Validación de Catálogos Maestros de RRHH:

/*****
*****/

TITULO: Validación de Catálogos Maestros de RRHH
OBJETIVO: Verificar la correcta carga de datos en los catálogos fundamentales
antes de iniciar procesos de importación de empleados o reportes.

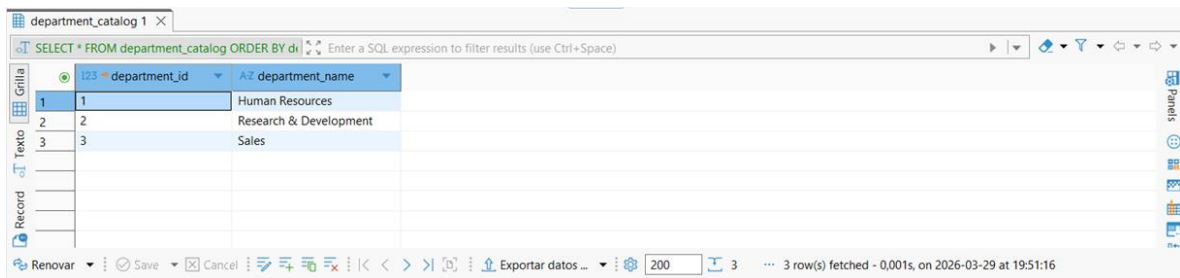
DESCRIPCION: Este script ejecuta tres consultas independientes para listar y ordenar los catálogos de departamentos, roles de trabajo y campos educativos, permitiendo al usuario asegurar la integridad de los datos referenciales (catálogos).

*****/
*****/

-- BLOQUE 1: Validación de Departamentos

-- Propósito: Visualizar la lista de departamentos activos para confirmar su existencia y correcta ortografía.

SELECT *
FROM department_catalog
ORDER BY department_name;-- Ordena alfabéticamente para facilitar la lectura.

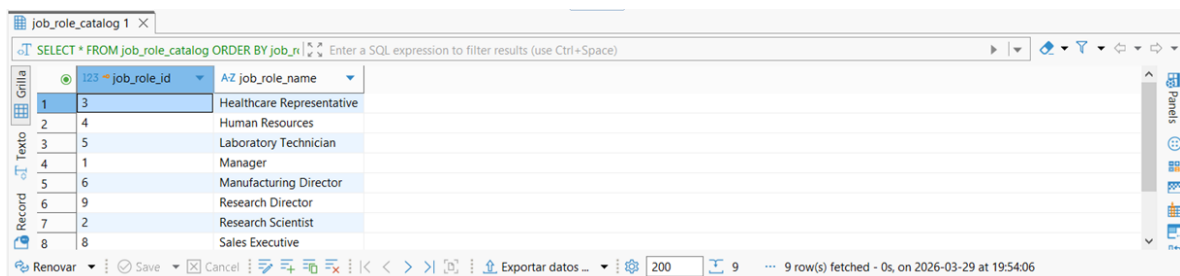


department_id	department_name
1	Human Resources
2	Research & Development
3	Sales

-- BLOQUE 2: Validación de Roles de Trabajo

-- Propósito: Revisar el catálogo de puestos/roles para asegurar que las denominaciones sean correctas.

SELECT *
FROM job_role_catalog
ORDER BY job_role_name;-- Ordena alfabéticamente por nombre del puesto.

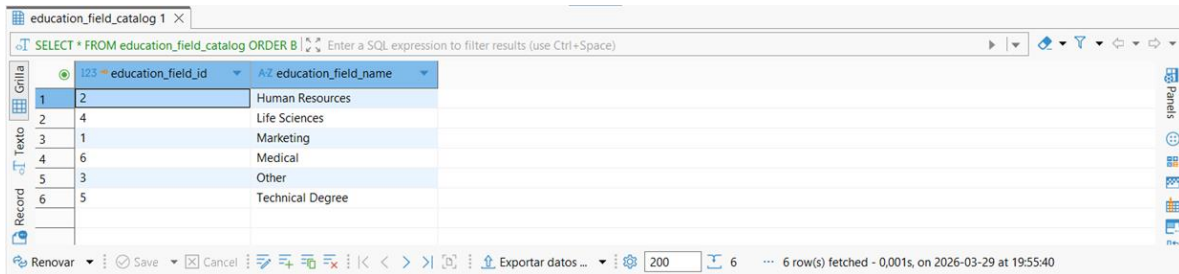


job_role_id	job_role_name
3	Healthcare Representative
4	Human Resources
5	Laboratory Technician
1	Manager
6	Manufacturing Director
9	Research Director
2	Research Scientist
8	Sales Executive

-- BLOQUE 3: Validación de Campos Educativos

-- Propósito: Confirmar la lista de áreas de estudio o campos educativos disponibles.

SELECT *
FROM education_field_catalog
ORDER BY education_field_name;-- Ordena alfabéticamente por nombre del campo educativo.



education_field_id	education_field_name
2	Human Resources
4	Life Sciences
1	Marketing
6	Medical
3	Other
5	Technical Degree

Paso 11: Creación de Tabla Maestra de Empleados Final:

/*****

TÍTULO: Preparación y Enriquecimiento de Datos de Empleados
OBJETIVO: Crear una tabla de trabajo definitiva y vincularla con catálogos.

DESCRIPCIÓN:

Este script realiza dos acciones principales:

1. Crea una copia de seguridad/trabajo de los datos maestros de empleados.
2. Añade nuevas columnas para vincular (hacer relaciones) al empleado con

tablas maestras de departamentos, puestos y áreas de estudio, garantizando

la integridad de los datos mediante restricciones (Foreign Keys).

*****/

*****/

--

=====

-- 1. CREACIÓN DE LA TABLA DE TRABAJO

--

=====

-- Se crea la tabla 'employee_final' basándose en 'employee_master_data'.

-- 'IF NOT EXISTS' evita errores si el script se ejecuta más de una vez.

CREATE TABLE IF NOT EXISTS employee_final AS

SELECT *

FROM employee_master_data; -- Copia toda la estructura y datos de la tabla maestra.

Estadísticas 1 X	
Name	Value
Updated Rows	1470
Execute time	0,016s
Start time	Sun Mar 29 20:43:24 CST 2026
Finish time	Sun Mar 29 20:43:24 CST 2026
Query	-- =====
	-- 1. CREACIÓN DE LA TABLA DE TRABAJO
	-- =====
	-- Se crea la tabla 'employee_final' basándose en 'employee_master_data'.
	-- 'IF NOT EXISTS' evita errores si el script se ejecuta más de una vez.
	CREATE TABLE IF NOT EXISTS employee_final AS
	SELECT *
	FROM employee_master_data
CST es Editable Inserción inteligente 19 : 7 : 1080 Set: 0 0	

--

=====

-- 2. ENRIQUECIMIENTO Y RELACIONES (ALTER TABLE)

--

=====

-- Modificamos la tabla 'employee_final' para añadir columnas de relación.

ALTER TABLE employee_final

-- Añade columna para el departamento y lo vincula con el catálogo de departamentos.

ADD COLUMN department_id INT REFERENCES
department_catalog(department_id),

-- Añade columna para el puesto y lo vincula con el catálogo de roles/puestos.

ADD COLUMN job_role_id INT REFERENCES
job_role_catalog(job_role_id),

-- Añade columna para el área educativa vinculada al catálogo de estudios.

ADD COLUMN education_field_id INT REFERENCES
education_field_catalog(education_field_id);

Estadísticas 1 X	
Name	Value
Updated Rows	0
Execute time	0,015s
Start time	Sun Mar 29 20:52:55 CST 2026
Finish time	Sun Mar 29 20:52:55 CST 2026
Query	-- =====
	-- 2. ENRIQUECIMIENTO Y RELACIONES (ALTER TABLE)
	-- =====
	-- Modificamos la tabla 'employee_final' para añadir columnas de relación.
	ALTER TABLE employee_final
	-- Añade columna para el departamento y lo vincula con el catálogo de departamentos.
	ADD COLUMN department_id INT REFERENCES department_catalog(department_id),
	-- Añade columna para el puesto y lo vincula con el catálogo de roles/puestos.
CST es Editable Inserción inteligente 29 : 12 : 1750 Set: 0 0	

```

/*****
*****
TITULO: Actualización de llaves foráneas en tabla de empleados
OBJETIVO: Normalizar la tabla 'employee_final' sustituyendo nombres
de texto
        por sus correspondientes IDs (llaves primarias) de tablas de
catálogo.
DESCRIPCIÓN: El script toma los valores de texto (departamento,
puesto, campo de
        estudio) de la tabla 'employee_final', los busca en sus
respectivos
        catálogos y actualiza la tabla principal con el ID numérico
correspondiente para mejorar la integridad referencial.
*****/

```

```

-- Actualizar la tabla final de empleados con los IDs correctos de los
catálogos
UPDATE employee_final e
SET
    -- Asigna el ID del departamento buscando por nombre
    department_id = d.department_id,
    -- Asigna el ID del puesto de trabajo buscando por nombre
    job_role_id = jr.job_role_id,
    -- Asigna el ID del campo educativo buscando por nombre
    education_field_id = ef.education_field_id
    -- Define las tablas de catálogo que actúan como fuente de
    búsqueda
FROM
    department_catalog d,
    job_role_catalog jr,
    education_field_catalog ef
-- Relaciona las tablas de catálogo con los textos descriptivos de la
tabla de empleados
WHERE e.department = d.department_name-- Coincidencia de
departamento
    AND e.job_role = jr.job_role_name-- Coincidencia de puesto (rol)
    AND e.education_field = ef.education_field_name;-- Coincidencia
de campo de estudio

```


Estadísticas 1	
Name	Value
Updated Rows	1470
Execute time	0,36s
Start time	Sun Mar 29 23:25:35 CST 2026
Finish time	Sun Mar 29 23:25:35 CST 2026
Query	-- Actualizar la tabla final de empleados con los IDs correctos de los catálogos
	UPDATE employee_final e
	SET
	-- Asigna el ID del departamento buscando por nombre
	department_id = d.department_id,
	-- Asigna el ID del puesto de trabajo buscando por nombre
	job_role_id = jr.job_role_id,
	-- Asigna el ID del campo educativo buscando por nombre
	CST es Editable Inserción inteligente 55 : 4 : 3180 Set: 0 0

Paso 12: Limpieza de la tabla final de empleados:

```

/*****
*****/

```

TITULO: Limpieza de la tabla final de empleados

OBJETIVO: Optimizar la estructura de la tabla 'employee_final' eliminando

columnas de texto redundantes que ya han sido procesadas o mapeadas

a valores numéricos/categorías, mejorando el rendimiento y reduciendo

el espacio de almacenamiento.

DESCRIPCION: Se eliminan las columnas 'department', 'job_role' y 'education_field'

debido a que la información ya se encuentra normalizada en otras

tablas o columnas dentro de la base de datos.

```

*****/

```

-- Eliminamos las columnas de texto originales

-- Esta sentencia modifica la estructura de la tabla 'employee_final' para borrar

-- los campos de texto que ya no son necesarios en el análisis.

ALTER TABLE employee_final

DROP COLUMN department, -- Elimina la columna con el nombre del departamento

DROP COLUMN job_role, -- Elimina la columna con el nombre del puesto

DROP COLUMN education_field; -- Elimina la columna con el área de estudio

Estadísticas 1 X	
Name	Value
Updated Rows	0
Execute time	0.007s
Start time	Sun Mar 29 23:32:57 CST 2026
Finish time	Sun Mar 29 23:32:57 CST 2026
Query	
-- Eliminamos las columnas de texto originales	
-- Esta sentencia modifica la estructura de la tabla 'employee_final' para borrar	
-- los campos de texto que ya no son necesarios en el análisis.	
ALTER TABLE employee_final	
DROP COLUMN department, -- Elimina la columna con el nombre del departamento	
DROP COLUMN job_role, -- Elimina la columna con el nombre del puesto	
DROP COLUMN education_field	

Paso 13: Validación de Estructura de Empleados:

```

/*
=====
===
TÍTULO: Validación de Estructura de Empleados
OBJETIVO: Verificar la correcta integración de datos.
DESCRIPCIÓN: Consulta de muestra para validar que la tabla
final de empleados esté correctamente vinculada con sus
catálogos (departamento, puesto, educación), mostrando
datos clave de rendimiento e ingresos.

=====
=== */

-- Seleccionamos las columnas relevantes para la validación:
-- Número de empleado, departamento, puesto, nivel educativo,
-- ingresos mensuales y calificación de desempeño.
SELECT e.employee_number,
       d.department_name,
       jr.job_role_name,
       ef.education_field_name,
       e.monthly_income,
       e.performance_rating
-- Definimos la tabla principal (empleados) como 'e'
FROM employee_final e
-- Unimos con catálogo de departamentos para obtener el nombre
JOIN department_catalog d ON e.department_id = d.department_id
-- Unimos con catálogo de puestos para obtener el nombre del rol
JOIN job_role_catalog jr ON e.job_role_id = jr.job_role_id
-- Unimos con catálogo de estudios para obtener el área educativa
JOIN education_field_catalog ef ON e.education_field_id =
ef.education_field_id
-- Limitamos el resultado a 10 registros para una revisión rápida
LIMIT 10;

```

employee_final(+) 1 X							
SELECT e.employee_number, d.department_name, j.job_role_name, e.education_field_name, e.monthly_income, e.performance_rating							
1	23	Sales	Manager	Life Sciences	15.427	3	
2	1	Sales	Sales Executive	Life Sciences	5.993	3	
3	2	Research & Development	Research Scientist	Life Sciences	5.130	4	
4	4	Research & Development	Laboratory Technician	Other	2.090	3	
5	5	Research & Development	Research Scientist	Life Sciences	2.909	3	
6	7	Research & Development	Laboratory Technician	Medical	3.468	3	
7	8	Research & Development	Laboratory Technician	Life Sciences	3.068	3	
8	10	Research & Development	Laboratory Technician	Medical	2.670	4	

Paso 14. Controles de la calidad de los datos (employee_final):

- Confirma que los datos se cargaron correctamente:

```

/*****

```

```

*****/

```

Título: Visualización Preliminar de Empleados

Objetivo: Obtener una muestra rápida de los datos almacenados en la tabla final

de empleados.

Descripción: Este script selecciona todos los campos de la tabla 'employee_final'

y limita el resultado a los primeros 10 registros para verificar la estructura y calidad de los datos sin cargar toda la tabla.

```

*****/

```

```

*****/

```

-- Selecciona todas las columnas (*) de la tabla principal de empleados

SELECT *

-- Indica la fuente de datos (tabla employee_final)

FROM employee_final

-- Limita la visualización solo a los primeros 10 registros

LIMIT 10;

employee_final 1 X											
SELECT * FROM employee_final LIMIT 10											
1	53	Female	Married	Y	No	Travel_Rarely	2	4	23	1	
2	41	Female	Single	Y	Yes	Travel_Rarely	1	2	1	1	
3	49	Male	Married	Y	No	Travel_Frequently	8	2	2	1	
4	37	Male	Single	Y	Yes	Travel_Rarely	2	1	4	1	
5	33	Female	Married	Y	No	Travel_Frequently	3	1	5	1	
6	27	Male	Married	Y	No	Travel_Rarely	2	1	7	1	
7	32	Male	Single	Y	No	Travel_Frequently	2	1	8	1	
8	59	Female	Married	Y	No	Travel_Rarely	3	1	10	1	
9	30	Male	Divorced	Y	No	Travel_Rarely	24	1	11	1	
10	38	Male	Single	Y	No	Travel_Frequently	23	3	12	1	

- Detección de registros duplicados:

```

/*****
*****
Título: Detección de Empleados Duplicados
Objetivo: Identificar inconsistencias en la base de datos de empleados.
Descripción: Este script analiza la tabla 'employee_final' para encontrar
números de empleado que aparecen más de una vez, lo que
indica
registros duplicados que deben ser revisados.
*****/

```

SELECT

```

-- Selecciona el número de empleado para identificarlo
employee_number,
-- Cuenta cuántas veces aparece cada número de empleado
COUNT(*) AS repeticiones
-- Define la tabla de origen de los datos
FROM employee_final
-- Agrupa los resultados por número de empleado para poder contarlos
GROUP BY employee_number
-- Filtra y muestra solo aquellos grupos con más de un registro
(duplicados)
HAVING COUNT(*) > 1;

```

employee_number	repeticiones
123	123 repeticiones

○ Identificación de Valores Nulos en Columnas Críticas:

```

/*****
*****
TITULO: Consolidación y Validación de Empleados Activos
OBJETIVO: Obtener una vista completa de los empleados, uniendo
información
de departamentos y puestos, asegurando la calidad de los datos.
DESCRIPCIÓN: Este script combina la tabla principal de empleados
(employee_final)
con los catálogos de departamentos y puestos para obtener
nombres

```

legibles en lugar de IDs. Además, filtra los registros para asegurar que la información tenga un mínimo de calidad (datos esenciales).

```
*****  
*****/
```

SELECT

```
e.employee_number,  
d.department_name,  
jr.job_role_name,  
e.monthly_income
```

FROM employee_final e

-- Une la tabla de empleados con el catálogo de departamentos para obtener el nombre del área

```
JOIN department_catalog d ON e.department_id = d.department_id
```

-- Une con el catálogo de puestos para obtener el nombre del rol laboral

```
JOIN job_role_catalog jr ON e.job_role_id = jr.job_role_id
```

-- CRITERIOS DE VALIDACIÓN: Filtra para mantener registros con información mínima necesaria

WHERE

-- 1. Verifica que el empleado tenga un número de identificación asignado

```
e.employee_number IS NOT NULL OR
```

-- 2. Verifica que el empleado esté vinculado a un departamento conocido

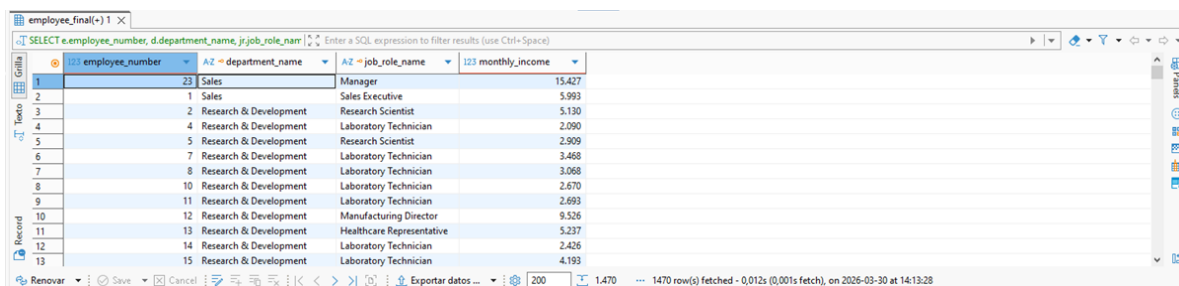
```
d.department_name IS NOT NULL OR
```

-- 3. Verifica que el empleado tenga un puesto de trabajo definido

```
jr.job_role_name IS NOT NULL OR
```

-- 4. Verifica que exista información sobre su salario mensual

```
e.monthly_income IS NOT NULL;
```



employee_number	department_name	job_role_name	monthly_income
23	Sales	Manager	15427
1	Sales	Sales Executive	5.993
2	Research & Development	Research Scientist	5.130
3	Research & Development	Laboratory Technician	2.090
4	Research & Development	Research Scientist	2.909
5	Research & Development	Laboratory Technician	3.468
6	Research & Development	Laboratory Technician	3.068
7	Research & Development	Laboratory Technician	2.670
8	Research & Development	Laboratory Technician	2.693
9	Research & Development	Manufacturing Director	9.526
10	Research & Development	Healthcare Representative	5.237
11	Research & Development	Laboratory Technician	2.426
12	Research & Development	Laboratory Technician	4.193
13	Research & Development	Laboratory Technician	4.193

○ Auditoría de Integridad (Valores fuera de rango o vacíos):

```
/*****  
*****/
```

TITULO: Auditoría de Datos de Empleados - Reporte de Errores

OBJETIVO: Identificar registros de empleados que contienen información incompleta

o ilógica en los campos de departamento y sueldo mensual.

DESCRIPCIÓN: Este script consulta la tabla final de empleados para localizar

filas donde el departamento esté vacío o el ingreso mensual sea cero o negativo, permitiendo la depuración de datos.

```
*****  
*****/
```

SELECT

e.employee_number, -- Número de identificación único del colaborador

d.department_name, -- Nombre del área de trabajo

e.monthly_income-- Salario mensual registrado

FROM employee_final e-- Tabla principal que contiene la información de los empleados

JOIN department_catalog d ON e.department_id = d.department_id--

Vincula el empleado con su departamento

-- CRITERIOS DE ERROR: Se seleccionan registros que cumplen condiciones ilógicas

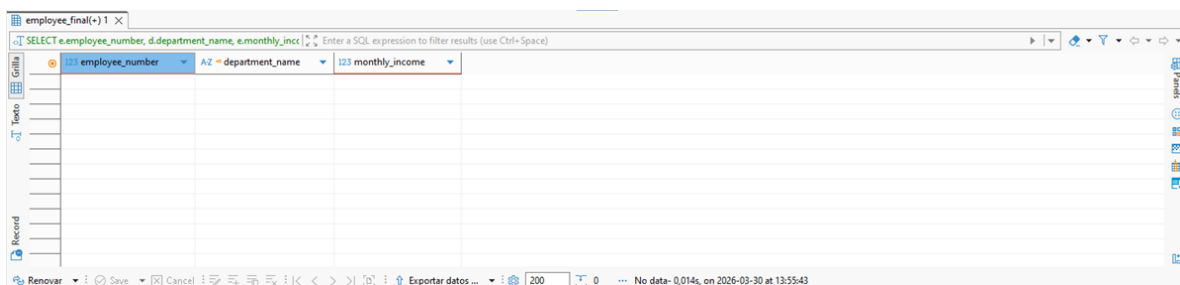
WHERE

-- 1. Identificar departamentos vacíos (texto sin contenido, por ejemplo: ")

d.department_name= " OR

-- 2. Identificar salarios no válidos (cero o negativos)

e.monthly_income <= 0;



○ Conteo total de colaboradores en la tabla principal:

```
/*****  
*****
```

TITULO: Conteo Total de Empleados

OBJETIVO: Obtener el número total de registros en la tabla de empleados.

DESCRIPCION: Este script realiza una consulta rápida para conocer el volumen actual de empleados censados en el sistema.

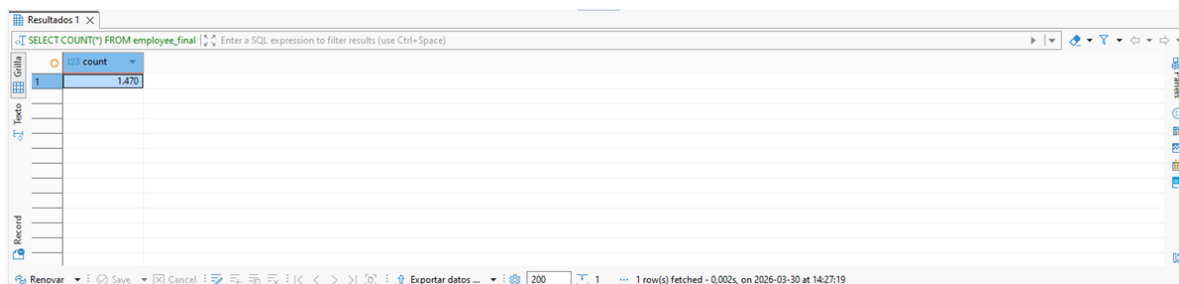
```
*****  
*****/
```

SELECT

COUNT(*) -- Cuenta el total de filas (registros) en la tabla, incluyendo nulos

FROM

employee_final;-- Especifica la tabla principal que contiene el censo de empleados



The screenshot shows a database query result window titled 'Resultados 1'. The SQL query entered is 'SELECT COUNT(*) FROM employee_final'. The result is displayed in a table with one row and one column labeled 'count', showing the value '1,470'. The status bar at the bottom indicates '1 row(s) fetched - 0,002s, on 2026-03-30 at 14:27:19'.

count
1,470

- **Distribución de la plantilla por departamento:**

```
/*****  
*****
```

TITULO: Reporte de Conteo de Empleados por Departamento

OBJETIVO: Obtener el número total de empleados que trabajan en cada área de la empresa.

DESCRIPCIÓN: Este script consulta la tabla de empleados, la relaciona con el

catálogo de departamentos para obtener el nombre real de cada área,

y realiza un conteo agrupado para mostrar el total de personal por

departamento.

```
*****  
*****/
```

SELECT

-- Selecciona el nombre del departamento de la tabla relacionada (d)

d.department_name,

-- Cuenta el total de empleados (filas) agrupados por departamento

COUNT(*) AS total_empleados

-- Selecciona la tabla de empleados como fuente principal (alias e)

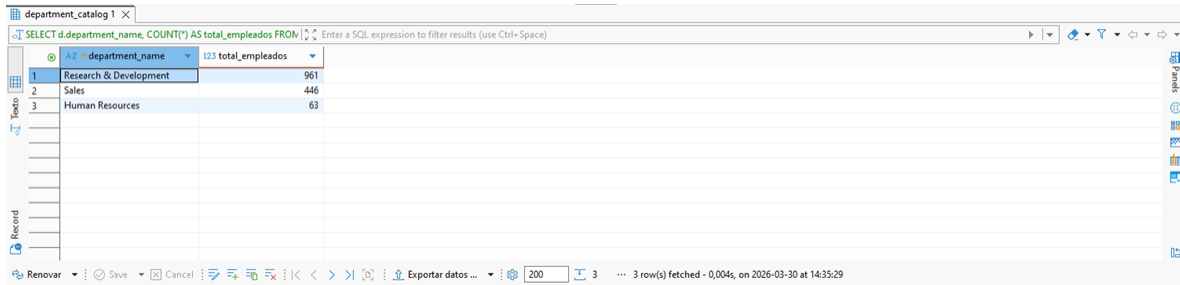
FROM employee_final e

-- Une con el catálogo de departamentos (alias d) usando el ID compartido

JOIN department_catalog d ON e.department_id = d.department_id

-- Agrupa el conteo anterior según el nombre del departamento

GROUP BY d.department_name;



The screenshot shows the DBeaver SQL editor with a query window titled 'department_catalog 1'. The query is: `SELECT d.department_name, COUNT(*) AS total_empleados FROM employee_final e JOIN department_catalog d ON e.department_id = d.department_id GROUP BY d.department_name;`. The results are displayed in a table with two columns: 'department_name' and 'total_empleados'. The data is as follows:

department_name	total_empleados
Research & Development	961
Sales	446
Human Resources	63

- **Pitfall común:** olvidar **GROUP BY** cuando usas agregaciones.
- **Check de Ingeniería:** validar que los resultados coinciden con expectativas (ej. número total de empleados).

Paso 15. Construir Esquema ER (Entity-Relationship) en DBeaver

En **DBeaver**:

Opción 1:

- Vaya al **Explorador de proyectos** (normalmente a la izquierda).
- Haga **clic derecho** en la carpeta **Diagramas** (Diagrams).
- Seleccione **Crear nuevo diagrama**.
- Seleccionar **Base de Datos**: Póngale un **nombre** al diagrama (ej. **ibm_hr_erd**) y busque la base de datos **ibm_hr**.
- Selecciona la tabla **employee_master_data**.
- **Clic en Finalizar**.
- También puede **arrastrar y soltar tablas** directamente desde el navegador de bases de datos hacia el **lienzo** del diagrama.
- **Generación Automática**: DBeaver visualizará automáticamente las **relaciones** existentes basadas en las **claves foráneas (FK)** y **primarias (PK)**.
- **Exporta** como **imagen** o **PDF** para tu portafolio.

employee_master_data
123 employee_number
123 age
AZ gender
AZ marital_status
AZ over18
AZ attrition
AZ business_travel
123 distance_from_home
AZ department
AZ job_role
123 job_level
123 employee_count
123 monthly_income
123 monthly_rate
123 daily_rate
123 hourly_rate
123 percent_salary_hike
123 stock_option_level
123 education
AZ education_field
123 training_times_last_year
123 environment_satisfaction
123 job_involvement
123 job_satisfaction
123 performance_rating
123 relationship_satisfaction
123 work_life_balance
123 num_companies_worked
123 total_working_years
123 years_at_company
123 years_in_current_role
123 years_since_last_promotion
123 years_with_curr_manager
AZ over_time
123 standard_hours

Opción 2:

- Vaya al **Navegador de Base de Datos** (normalmente a la izquierda).
- Busque la base de datos **ibm_hr**.
- Doble **Clic** en la tabla **employee_master_data**.
- Seleccione la carpeta **Diagramas** (Diagrams).

employee_master_data
123 employee_number
123 age
AZ gender
AZ marital_status
AZ over18
AZ attrition
AZ business_travel
123 distance_from_home
AZ department
AZ job_role
123 job_level
123 employee_count
123 monthly_income
123 monthly_rate
123 daily_rate
123 hourly_rate
123 percent_salary_hike
123 stock_option_level
123 education
AZ education_field
123 training_times_last_year
123 environment_satisfaction
123 job_involvement
123 job_satisfaction
123 performance_rating
123 relationship_satisfaction
123 work_life_balance
123 num_companies_worked
123 total_working_years
123 years_at_company
123 years_in_current_role
123 years_since_last_promotion
123 years_with_curr_manager
AZ over_time
123 standard_hours

- **Blueprint lógico:** piensa en el **ER** como el **plano arquitectónico** del edificio de datos. Cada **columna** es un **pilar** que soporta **análisis futuros**.



Paso 16. Conexión Python-SQL

Integra **SQL** con **Python** para extracción inicial:

"""

TÍTULO: Extracción Modular de Datos desde PostgreSQL

OBJETIVO: Consultar la tabla maestra de empleados de forma segura y organizada.

DESCRIPCIÓN: El script utiliza variables de entorno para proteger las credenciales,

se conecta a la base de datos 'ibm_hr', extrae una muestra de empleados y gestiona automáticamente el cierre de la conexión para optimizar recursos.

```
"""
```

```
import os
import psycopg2 # Conector para comunicarse con PostgreSQL
import pandas as pd # Herramienta para analizar y organizar datos en tablas
from dotenv import load_dotenv # Carga variables secretas desde un archivo .env
```

```
# Bloque de Configuración: Cargamos las credenciales ocultas del sistema
load_dotenv()
```

```
def obtener_conexion_db():
```

```
    """
```

```
    Establece el puente de comunicación con el servidor.
```

```
    Usa 'os.getenv' para leer los datos de acceso sin exponerlos en el código.
```

```
    """
```

```
    return psycopg2.connect(
        user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'),
        host=os.getenv('DB_HOST'),
        port=os.getenv('DB_PORT'),
        database=os.getenv('DB_NAME')
    )
```

```
# Definimos cuántos registros queremos traer por defecto
limite = 10
```

```
def extraer_datos_empleados(limite):
```

```
    """
```

```
    Lógica principal para realizar la consulta y traer los datos a Python.
```

```
    """
```

```
    conn = None
```

```
    try:
```

```
        # Iniciamos la conexión llamando a la función anterior
```

```
        conn = obtener_conexion_db()
```

```
        # Definimos la instrucción SQL usando el límite solicitado
```

```
        query = f"""
```

```

SELECT *
FROM employee_master_data
LIMIT {limite}
"""

print("\n🚀 Iniciando ingesta PostgreSQL...")

# pandas ejecuta la consulta y transforma el resultado en una tabla
(DataFrame)
df = pd.read_sql(query, conn)

print("✅ ¡Datos cargados exitosamente!")
return df

except Exception as e:
    # Si algo falla (servidor caído, error de red), mostramos el motivo
    exacto
    print(f"❌ Error al conectar o procesar la base de datos: {e}")
    return None

finally:
    # Bloque de Cierre: Pase lo que pase, nos aseguramos de cerrar la
    conexión
    if conn:
        conn.close()

# Bloque de Ejecución: Punto de partida del programa
if __name__ == "__main__":
    # Llamamos a la función principal usando el límite definido arriba
    df_empleados = extraer_datos_empleados(limite)

    # Si la descarga fue exitosa, mostramos los resultados en pantalla
    if df_empleados is not None:
        print("\n Vista previa de los datos:")
        print(df_empleados.head())

```

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  HISTORIAL SQL  SUPERVISIÓN DE TAREAS  ...
🚀 Iniciando ingesta PostgreSQL...
✅ ¡Datos cargados exitosamente!

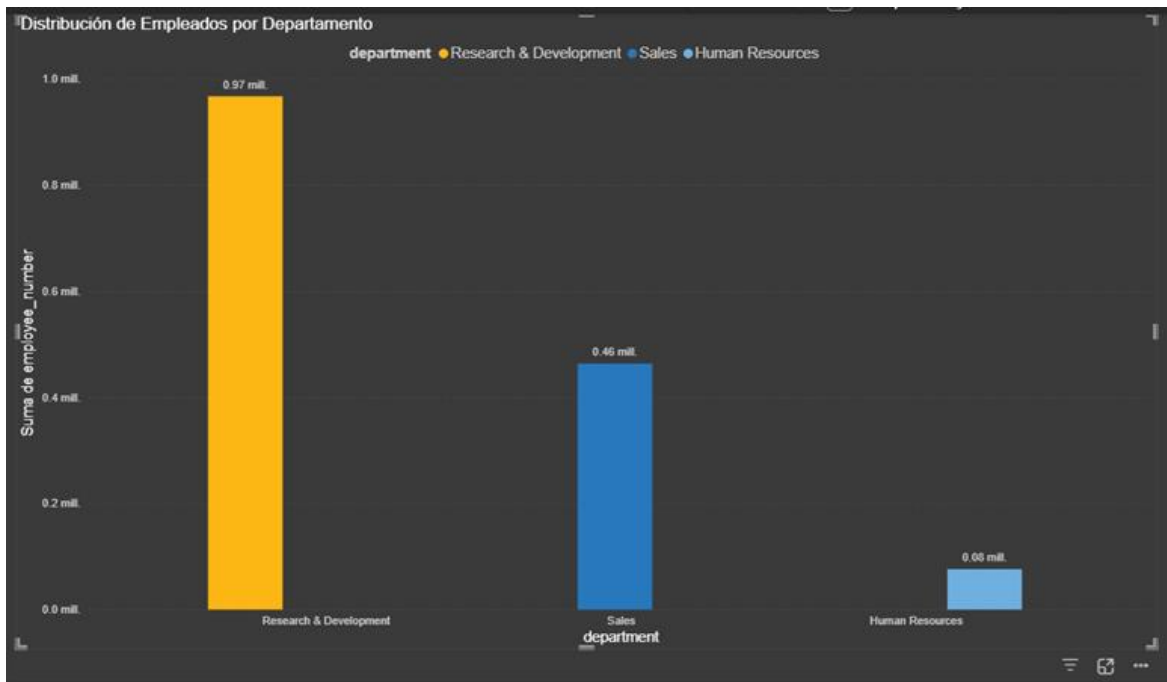
   age  gender  marital_status  over18  attrition  ...  years_in_current_role  years_since_last_promotion  years_with_curr_manager  over_time  standard_hours
0   41  Female      Single        Y        Yes  ...                4                0                5        Yes            80
1   49   Male      Married        Y        No   ...                7                1                7        No            80
2   37   Male      Single        Y        Yes  ...                0                0                0        Yes            80
3   33  Female      Married        Y        No   ...                7                3                0        Yes            80
4   27   Male      Married        Y        No   ...                2                2                2        No            80

[5 rows x 35 columns]
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code>

```

Paso 17. Conexión Power BI-SQL

- **Abrir Power BI Desktop:**
 - Localiza el ícono de **Power BI Desktop** en tu menú de inicio o barra de búsqueda.
 - Haz clic para abrir la aplicación.
 - Espera a que cargue la pantalla inicial con la opción de **Obtener datos**.
- **Seleccionar Fuente de Datos → PostgreSQL/SQL Server:**
 - En la pantalla principal, haz clic en **Obtener datos** (Get Data).
 - Se abrirá una ventana con múltiples opciones de fuentes.
 - En la opción **Base de Datos** selecciona **PostgreSQL**.
 - Haz clic en **Conectar**.
- **Configurar Servidor:**
 - En el cuadro de diálogo que aparece:
 - **Servidor:** escribe localhost (esto indica que la base está en tu propia máquina).
 - **Base de datos:** escribe **ibm_hr**.
 - Haz clic en **Aceptar**.
- **Autenticación:**
 - **Usuario:** postgres
 - **Contraseña:** la que definiste al instalar **PostgreSQL** (ej. SaPassword2026!).
 - Haz clic en **Conectar**.
- **Importar la Tabla:**
 - **Power BI** mostrará un navegador con las tablas disponibles en la base **ibm_hr**.
 - Selecciona la tabla que creaste (ej. employee_master_data).
 - Haz clic en **Cargar**.
 - **Power BI** importará los datos y los mostrará en el panel de campos.
- **Crear un Gráfico de Barras: Empleados por Department:**
 - En la vista de **Informe** (Report View), haz clic en el lienzo en blanco.
 - En el panel de **Visualizaciones**, selecciona el ícono de **Gráfico de barras agrupadas**.
 - Arrastra el campo department al eje **Categorías (Eje X)**.
 - Arrastra el campo employee_number al eje **Valores (Eje Y)**.
 - **Power BI** automáticamente contará el número de empleados por departamento.
 - **Ajusta el formato:**
 - **Título:** "Distribución de Empleados por Departamento".
 - **Colores:** selecciona paleta corporativa o estándar.
 - **Etiquetas de datos:** activa para mostrar los valores sobre cada barra.



- **Insights:**
 - **Predominio del Core de Innovación:** Con 961 empleados, el área de **Research & Development (R&D)** es el **core de innovación** y **ventaja competitiva**. Operativamente, gestionar la **retención de talento** aquí es crítico, ya que cualquier cambio en este grupo de trabajo supone el mayor **impacto financiero** para la empresa.
 - **Motor Comercial Consolidado:** El **área comercial** (446 personas) sostiene el **crecimiento escalable** de **Research & Development (R&D)**, actuando como un **go-to-market** consolidado que garantiza ingresos estables.
 - **Estructura Administrativa Esbelta:** El departamento de **HH.RR.** destaca por su **estructura esbelta** (63 profesionales), gestionando una ratio aproximada de 1:23. Esto refleja **eficiencia administrativa**, aunque sugiere **implementar automatización** para evitar la **saturación del equipo**.
- **Validación del Paso 7:**
 - **SQL:** confirmas que la tabla integral está disponible en la base **ibm_hr**.
 - **Power BI:** logras importar la tabla y visualizar un gráfico de barras.
 - **Resultado esperado:** un dashboard inicial que muestra la cantidad de empleados por departamento.